

Tool for AI Detection in Modern Academic Papers

Joseph Tully

2026-09-24

Contents

1 Purpose & Goals	3
1.a Goals Table	3
2 Data Set	3
2.a Info about Data Set	3
3 Toolkit	3
3.a Machine Learning	3
3.b Language Processing	3
3.c Interface	3
4 Primary Sketch	4
5 Methodology	5
5.a Features Implemented	5
6 Code Explanation	5
6.a Burstiness & Sentence Length	5
6.b Split Sentences	5
6.c Burstiness	5
6.d Verification	5
Appendix	6
A Abbreviations	6
B Split Sentences	7
C Burstiness	7

1 | Purpose & Goals

Can an AI detect other AI? We believe so. By combining features including sentence length, grammar, vocabulary richness, passive voice ratio, and reading level. Some of the features are stretch goals because with limited time and resources.

1.a | Goals Table

Features	Importance Rating (out of 5)	Difficulty (out of 5)
Grammar Detection	2	2
Sentence Length	4	1
Vocabulary Richness	5	3
Passiv Voice	3	2
Reading Level	1	4

2 | Data Set

The data set being used includes outliers within the dataset. The outliers should help the robustness of our model, when our model encounters data that isn't similar to the data it had been seeing, it will make a decision not based off recognizable patterns. By making our model able to identify these outlier accurately we can increase our confidence rating for the final product.

2.a | Info about Data Set

10,200 rows each representing a piece of writing. With around 10,000 pieces of writing our model will be well trained, and will learn to categorize from a diverse and complex mix of data. From the data we will collect certain metrics before beginning the training. While outliers will help our models they may also skew the data during training, therefore we have to process our dataset first. We will be making sure the *AI written* papers are actually written entirely by an artificial intelligence model, while the *human papers* do not use any AI in the writing of the paper. This process is known as data set cleaning. We will use previous ai detection models to help make sure our data set is properly organized/cleaned before we begin the training is started.

3 | Toolkit

3.a | Machine Learning

- PyTorch - neural-network classifier as an extension of the project
- Scikit-learn - preprocessing, model training, evaluation, and feature analysis
- NumPy - numerical operations
- Pandas - dataset cleaning, manipulation, and analysis

3.b | Language Processing

- spaCy - sentence segmentation, tokenization, parts of speech, other linguistic features
- NLTK - Additional text statistics
- PassivePy - Measurement of passive voice usage
- textStat - readability measures such as Flesch-Kincaid grade level

3.c | Interface

- Matplotlib - visualizations, graphs
- Gradio/Streamlit - Interactive demo

4 | **Primary Sketch**

There are several classifications we will be using for this project such as, logistic regression, random forest, and gradient boosting. These machine learning strategies will be implemented and compared. For us this is an opportunity to adjust the hyper parameters for our model, making it more efficient at learning as we go. The final deliverable will be a demo where a user can type out text or import a text file, to check for the use of ai within writing. After the text has been looked at by the model, it will be evaluated by the given parameters and judged. Once the model is done processing the text it will return a confidence score in real-time. The confidence score will return a result (ai/human) and an informational breakdown behind the reasoning of the confidence score.

5 | Methodology

5.a | Features Implemented

6 | Code Explanation

6.a | Burstiness & Sentence Length

In Appendix A you can see the list of abbreviations these abbreviations are potentially dangerous to our function. Initially we need to divide the text into sentences before we can count their length. When doing this we will split at 'end of sentence' punctuation(., ?, !, etc.). The danger comes when we encounter an abbreviation like "Dr. Smith" where the sentence would be split, "Dr." and "Smith". To avoid this problem this function was implemented in order to differentiate between abbreviation and the end of a sentence.

6.b | Split Sentences

To gain the sentence length for each sentence in a text file, the sentences need to be separated properly. This is done using the `split_sentences` function found in Appendix B. Split sentences will take in a list of sentences and looks for where characters are split with a space, `text = ".join(text.split())`. The function then filters through the splits for abbreviations, calling on Appendix A the abbreviations function. It can then repair the words and count them properly. This function returns a list of sentences from the buffer.

6.c | Burstiness

When feeding data into a model, it is important to not overcomplicate it. This simply causes the model to take longer and make more computations per task. To help this model will be given an arbitrary value representing the consistency between sentence lengths. This is calculated through the burstiness function in Appendix C. The function takes the word count per sentence as a list and feeds it into the `python statistics.pstdev()` and divides it by the mean of those same lengths. This gives us an arbitrary value, where the greater the value the more "Bursty" the sentence lengths are.

6.d | Verification

```
split_sentences
PASS simple
PASS all enders
PASS no punctuation
PASS empty
PASS whitespace
PASS title
PASS figure ref
PASS initials
PASS et al
PASS list marker
PASS real word no
PASS year at end
PASS number at end
PASS keeps final sentence
PASS keeps punctuation

count_words
PASS plain
```

```

PASS punctuation
PASS contraction
PASS drops numbers
PASS drops citation
PASS empty

burstiness - guard clause
PASS empty returns None
PASS too short returns None
PASS exactly five is scored

burstiness - anchors
PASS flat is zero (0.0000)
PASS uniform (0.1473)
PASS moderate (0.7634)
PASS varied (1.3757)
PASS extreme (1.8912)

burstiness - ordering
PASS strictly increasing

punctuation_usage
PASS empty
PASS none present
PASS one of each
PASS repeats
PASS ellipsis
PASS ignores others

proper_punctuation_usage
PASS all proper
PASS empty
PASS one lowercase
PASS all lowercase
PASS no final stop
PASS quote merges

```

Appendix

A | Abbreviations

```

ABBREVIATIONS = {
    "dr", "mr", "mrs", "ms", "prof", "sr", "jr", "st",
    "e.g", "i.e", "cf", "vs", "etc", "al", "fig", "eq", "vol",
    "jan", "feb", "mar", "apr", "jun", "jul", "aug", "sep", "oct", "nov", "dec",
}

_BOUNDARY = re.compile(r'(?<=[.!?])\s+')

_WORD = re.compile(r"[A-Za-z']+")

def _ends_inabbreviation(buffer: str) -> bool:
    stripped = buffer.rstrip('"\'')
    words = stripped.split()
    if not words:

```

```

        return False
last = words[-1].rstrip('.').lower()

if last in ABBREVIATIONS:
    return True
if len(last) == 1 and last.isalpha():
    return True
if re.fullmatch(r'\d+\.', stripped):
    return True
return False

```

B | Split Sentences

```

def split_sentences(text: str) -> list[str]:
    #I am going to split by 'oversplitting' then repairing the broken sentences
    text = " ".join(text.split())
    if not text:
        return []

    sentences = []
    buffer = ""

    for piece in _BOUNDARY.split(text):
        buffer = f"{buffer} {piece}".strip() if buffer else piece
        if _ends_inabbreviation(buffer):
            continue
        sentences.append(buffer)
        buffer = ""

    if buffer:
        sentences.append(buffer)

    return sentences

```

C | Burstiness

```

def burstiness(text: str) -> float | None:
    lengths = [count_words(s) for s in split_sentences(text)]
    lengths = [n for n in lengths if n > 0]
    if len(lengths) < 5:
        return None
    return statistics.pstdev(lengths) / statistics.fmean(lengths)

```